

CLAUDE CODE TIP SHEET

A three-page reference: keyboard shortcuts & setup, the complete slash-command catalog organized by workflow phase, and practical tips from the community.

KEYBOARD SHORTCUTS	
!	Bash mode prefix
@	Mention files / folders
\	Line break (backslash + Enter)
Esc	Interrupt Claude
Esc Esc	History navigation
Ctrl + R	Toggle full output / context
Ctrl + V	Paste image
Ctrl + L	Fix garbled terminal text
Shift + Tab	Auto-accept ("yolo mode")
Shift + Tab x2	Plan mode
#	Quick-add to CLAUDE.md
Cmd + Esc	Quick launch in IDEs

TERMINAL COMMANDS	
<code>npm install -g @anthropic-ai/claude-code</code>	Install Claude Code globally
<code>claude update</code>	Update to latest version
<code>claude</code>	Start an interactive session in the current dir
<code>claude -c · claude --resume</code>	Resume last session after a crash or close
<code>claude --debug</code>	Launch with debug logging enabled

CONFIGURATION	
<code>claude config list</code>	View all settings
<code>claude config set <key> <value></code>	Change a setting value
<code>claude config add <key> <value></code>	Append to an array setting
<code>claude config set -g theme dark</code>	Enable dark theme globally

KEY FILE LOCATIONS	
<code>~/.claude/settings.json</code>	User global settings
<code>.claude/commands/</code>	Custom slash commands
<code>.claude/rules/</code>	Coding standards & enforced processes
<code>~/.claude/projects/</code>	Per-project prompt & token logs

macOS / Linux paths shown · on Windows, swap ~ / for %USERPROFILE%\ and use backslashes

WORKFLOW AT A GLANCE	
01 FIRST SESSION IN A REPO	Generate a starter CLAUDE.md and refine it. Set up subagents, MCP servers, and approval rules the project will need. <code>/init</code> <code>/memory</code> <code>/mcp</code> <code>/agents</code> <code>/permissions</code>
02 DURING A TASK	Plan before big changes, adjust how much reasoning you're spending, and watch context. Use <code>/btw</code> for a quick aside that doesn't bloat history. <code>/plan</code> <code>/model</code> <code>/effort</code> <code>/context</code> <code>/compact</code> <code>/btw</code>
03 RUNNING WORK IN PARALLEL	Detach to a background agent, list what's running, or decompose a codebase-wide change into independent worktrees that each open a PR. <code>/background</code> <code>/tasks</code> <code>/batch</code> <code>/loop</code>
04 BEFORE YOU SHIP	See what changed, run an automated quality & efficiency pass, and get a deeper read-only review — locally or in the cloud. <code>/diff</code> <code>/simplify</code> <code>/review</code> <code>/security-review</code> <code>/ultrareview</code>
05 BETWEEN SESSIONS	Start fresh while keeping project memory. Resume by ID or name. Fork the conversation at a checkpoint. Pull a web session into your terminal. <code>/clear</code> <code>/resume</code> <code>/branch</code> <code>/teleport</code> <code>/remote-control</code>
06 WHEN SOMETHING IS WRONG	Roll code & conversation back to a checkpoint. Diagnose install or runtime issues. Report a bug with session context attached. <code>/rewind</code> <code>/doctor</code> <code>/debug</code> <code>/feedback</code>

CLAUDE.MD IN 60 SECONDS	
It ships in every system prompt	
So treat tokens like rent. Trim anything that doesn't change behavior — grep transcripts for verbatim lines and cut what never shows up.	
Use it as a pointer, not a manual	
A 3-line file that says "read README.md and docs/ before starting" beats a sprawling one. Use <code>@import</code> for deeper context.	
Quick-add with #	
Start a message with <code>#</code> + a rule and Claude writes it to CLAUDE.md. In	
<code>.claude/settings.json</code>	Project-level settings (committable)
<code>.claude/agents/</code>	Project subagents
<code>CLAUDE.md</code>	Always-on context, every session
<code>~/.claude/themes/</code>	Custom color themes

COMMANDS BY **WORKFLOW PHASE**

Every Claude Code slash command grouped by when you'll reach for it. Type `/` in a session to see the full picker; `/help` for inline docs.

01 SETUP & ONBOARDING

- `/init` Generate a starter **CLAUDE.md** for the project.
- `/memory` Edit **CLAUDE.md**, manage auto-memory entries.
- `/agents` Manage subagent configurations.
- `/mcp` Connect & authenticate MCP servers.
- `/permissions` Allow / ask / deny rules for tool use.
- `/add-dir <path>` Add a working directory for file access.
- `/team-onboarding` Generate a teammate setup guide from history.
- `/install-github-app` Set up Claude GitHub Actions for a repo.

02 DURING A TASK

- `/plan [desc]` Enter plan mode, optionally with a task.
- `/goal [cond]` Keep working across turns until condition met.
- `/model [name]` Select or change the model mid-session.
- `/effort [level]` Set reasoning level: low / med / high / xhigh / max.
- `/context [all]` Visualize what's eating your context window.
- `/compact [focus]` Summarize history to free context.
- `/btw <q>` Quick side question without bloating history.
- `/focus` Hide tool clutter; show only prompt + reply.
- `/fast [on|off]` Toggle fast mode.
- `/loop [int] [p]` Run a prompt repeatedly while session is open.

03 PARALLEL & BACKGROUND

- `/tasks` List and manage background tasks.
- `/background [p]` Detach session as a background agent. (`/bg`)
- `/batch <instr>` Decompose into 5–30 units in worktrees + PRs.
- `/autofix-pr [p]` Watch current branch's PR, push fixes on CI fail.

04 BEFORE YOU SHIP

- `/diff` Interactive viewer: git diff + per-turn diffs.
- `/review [PR]` Local pull-request review in this session.
- `/simplify [focus]` 3 parallel reviewers + auto-fix quality issues.
- `/security-review` Scan diff for injection, auth, data-exposure risks.
- `/ultrareview [PR]` Deep multi-agent review in a cloud sandbox.
- `/ultraplan <p>` Draft a plan remotely, review it, then execute.

05 BETWEEN SESSIONS

- `/clear [name]` New conversation, empty context. (`/reset`, `/new`)
- `/resume [id]` Resume by ID or name. (`/continue`)
- `/branch [name]` Fork the conversation at this point. (`/fork`)
- `/rename [name]` Name the session; shows on prompt bar.
- `/export [file]` Export conversation as plain text.
- `/teleport` Pull a Claude Code on the web session into terminal.
- `/remote-control` Drive this session from claude.ai. (`/rc`)
- `/desktop` Continue session in the Desktop app.
- `/copy [N]` Copy last reply (or Nth-latest) to clipboard.
- `/recap` One-line summary of the current session.

06 WHEN SOMETHING'S WRONG

- `/rewind` Undo to a checkpoint. (`/checkpoint`, `/undo`)
- `/doctor` Verify install + settings; press `f` to auto-fix.
- `/debug [desc]` Enable debug logging & analyze the log.
- `/feedback [r]` Submit a bug report with session context.
- `/release-notes` Interactive changelog viewer.

07 SETTINGS & CUSTOMIZATION

- `/config` Open Settings UI: theme, model, output style. (`/settings`)
- `/theme` Pick / create a color theme (auto, light, dark).
- `/color [c]` Set prompt-bar color for this session.
- `/statusline` Configure the status line at the prompt.
- `/keybindings` Open / create keybindings config.
- `/tui [mode]` Switch terminal UI renderer (default / fullscreen).
- `/voice [mode]` Voice dictation: hold / tap / off.
- `/sandbox` Toggle sandbox mode (supported platforms).
- `/ide` Manage IDE integrations & status.
- `/chrome` Configure Claude in Chrome settings.

08 SYSTEM & INFO

- `/help` Show help and available commands.
- `/status` Version, model, account, connectivity.
- `/usage` Cost, plan limits, activity. (`/cost`, `/stats`)
- `/insights` Report: project areas, patterns, friction points.
- `/skills` List skills; press `t` to sort by tokens.
- `/hooks` View hook configs for tool events.
- `/plugin` Manage Claude Code plugins.
- `/reload-plugins` Apply plugin changes without restart.
- `/login` Sign in to your Anthropic account.
- `/logout` Sign out.
- `/exit` Exit the CLI. (`/quit`)

A command is only recognized at the **start** of your message.

PRO TIP Text after the name is passed as **arguments**. `<arg>` = required, `[arg]` = optional.

WORKING **SMARTER** WITH CLAUDE CODE

Hard-won patterns from the Claude Code community — ordered roughly from “use today” to “use when scaling.”

RULE 0 Talk to Claude like a partner, not a tool. Quality of interaction sets the ceiling for quality of output. Pair it with a written plan, one task per session, and a habit of asking for receipts.

01 MODEL & SESSION

› Strategic model use

Use the **most capable** model for planning (low/medium thinking) and a **smaller, faster** model for implementation — small models follow a clear plan well.

Tell the planner to act as *director* and delegate routine work down a tier.

› Switch mid-session in peak hours

When responses slow during US business hours, run `/model` and pick a lighter tier mid-conversation. Latency usually drops immediately.

› One task, one session

Plan epics & stories *outside* Claude Code. Bring exactly one task in. **New task** → **new session, always**. Over 30 min on unrelated work? Kill it — context is working against you.

02 CLAUDE.MD

› Quick-add with `#`

At the start of any message, type `#` + a rule. Claude writes it to CLAUDE.md. In newer versions: just say “remember X.”

› Use it as a pointer

A 3-line file that says “read README.md and docs/ before starting” beats a sprawling one. `@import` directly, or `@AGENTS.md` on line one.

› Trim ruthlessly

CLAUDE.md ships in **every** system prompt. Grep recent transcripts for lines verbatim from the file — cut anything that never shows up. Often halves the file with zero behavior change.

03 WORKFLOW & PLANNING

› Ask for receipts

After anything bigger than a quick patch, demand: *what changed, files touched, commands run, what's still broken, exact next step*. Forces Claude to close its own loop and catches “I’ll leave that for later” drift.

Next level: write the next task as a single imperative sentence in CLAUDE.md before closing the session.

› Ask deep clarifying questions

Add to requests: “*Ask me clarifying questions, but gather enough information first. Make them deep and thought-provoking — reveal blind spots, skip the obvious.*”

› Voice + clipboard hooks

On macOS, hook the `say` command on Stop — a one-sentence announcement is enough context to navigate back between parallel sessions.

Ask agents to write handoff prompts directly to your **clipboard**.

04 CONTEXT & MEMORY

› Never work in memory only

Write plan, progress, and current state to local **.md** files every turn. Claude crashes — if it isn’t on disk, you rebuild from scratch.

› Survive compaction

At 90% context: ask Claude to write a Handoff doc + update LESSONS.md / BACKLOG.md, then manually `/compact`, then have it re-read those docs.

› Manual summaries beat auto-compact

Turn auto-compact off. Cached context is usually cheaper than a fresh start. If you accidentally fill the window, step back and write the summary file first — without reading anything new.

05 DEBUGGING & QUALITY

› Control how Claude diagnoses

Otherwise it will spend hours on a basic bug. Direct it to: *think critically* → *add debug code* → *read logs* → *analyze*. Bake the steps into a skill.

› Don’t anchor what it knows

~80% of prompt tokens get burned re-explaining things the model already knows. Reserve CLAUDE.md for: novel problems, personal style, and post-training context.

› Lint & enforce in code

Run linters so the codebase stays maintainable. A rule in an agent system **is not a rule unless enforced by code** — use hooks, not hope.

06 TOKENS & COST

› Monitor what you actually use

Mine `~/ .claude/projects/` for prompts & token use (raw data behind `/insights`). Combine with `ccusage` to track cache vs spend.

› Trim explore-agent context

Explore agents load **every** MCP server & skill into context on spawn. If they boot above 15–20k tokens, prune.

07 SCALING UP

› Parallel agents with `/batch`

Make one session the *supervisor*. It decomposes a big change into independent units, spawns one background agent per unit in its own worktree, and opens PRs. Pair with thorough design docs.

› Internal marketplace

Host shared skills, agents, and hooks in a company git repo. Mark source as git, mark the plugin auto-updating, bump semver on each PR.

› Use `.claude/rules/`

The right home for coding standards, process gates, and evidence-required guards. Skills, agents, and rules are all just prompts — institutionalize good practice there instead of re-typing it.